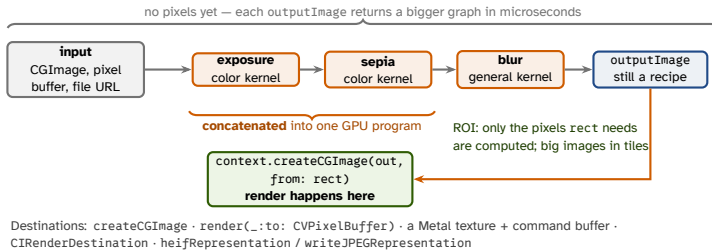
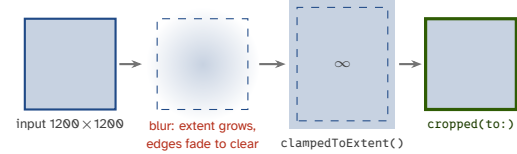


In one line: A `CIImage` is a **recipe, not pixels**: applying a `CIFilter` only extends a lazy graph. Nothing is computed until a `CITexture` renders a rectangle of it — then Core Image walks the graph *backwards* from that rectangle (region of interest), **concatenates** adjacent kernels into as few GPU (Metal) passes as it can, and writes the result to a `CGImage`, pixel buffer, texture or file. Contexts are **expensive** — **make one and reuse it**; contexts and images are immutable and thread-safe, filters are mutable and are not.

A. Build a recipe (free) → render a rectangle (all the work)



B. Extent — the blur trap



Outside its extent an image is clear black. Clamp (repeat the edge pixels, infinite extent) before the blur, crop back to input.extent after. Generators (`CIColorGenerator`, checkerboard) are infinite too: rendering `.infinite` fails — always crop.

How it works

- Inputs:** `CIImage(cgImage:)`, `(cvPixelFormat:)` (camera frames, no copy), `(image:)` from `UIImage` — which drops `imageOrientation`: apply `.oriented(_)`. Origin is **bottom-left**, like Quartz.
- Two filter APIs:** stringly `CIFilter(name: "CIGaussianBlur") + setValue(_:forKey: kCIInputImageKey)` (typos = runtime nil), or typed `import CoreImage.CIFilterBuiltins (iOS 13) → CIFilter.gaussianBlur()` with `inputImage / radius` properties.
- Lazy + concatenated:** chains of per-pixel (colour) kernels fuse into one program — no intermediate texture per step; a blur or warp needs neighbours, so it may force one.
- CITexture** owns an `MTLCommandQueue`, the compiled-kernel cache and intermediate buffers. Apple: *one per view that renders, or per background task* — never per image or per frame. `CITexture(mtlDevice:) / (mtlCommandQueue:)` shares your Metal pipeline; `.useSoftwareRenderer: true` forces the (slow) CPU path.
- Colour:** inputs are converted into the **working space** — default **extended sRGB, linear gamma**, so maths happens in linear light and values beyond 0–1 survive; output converts to the destination's space. Masks, depth, data: `.workingColorSpace: NSNull()` turns colour management off.
- Custom kernels** are Metal (MSL): `CIColorKernel` (one pixel in, one out), `CIWarpKernel` (returns the *source coordinate*), `CIBlendKernel`, general `CIKernel` (samplers + an ROI callback). Build: *Other Metal Compiler Flags* `-fcikernel`, user setting `MTLLINKER_FLAGS = -cikernel`. String kernels (`CIKernel(source:)`) are **deprecated since iOS 12**. MPS or a model inside the graph: `CIImageProcessorKernel`.
- Vision hand-off:** the same `CIImage` feeds `VNImageRequestHandler(ciImage:orientation:)`; results are **normalised** (0–1), **bottom-left** origin — convert with `VNImageRectForNormalizedRect`, then e.g. `pixellate` only the face rect and composite it back.

Example — one shared context, typed filters

```
import CoreImage.CIFilterBuiltins
let context = CITexture() // ONE, app-wide or per view
func stylize(_ cg: CGImage) -> CGImage? {
    let input = CIImage(cgImage: cg)
    let blur = CIFilter.gaussianBlur() // typed API, iOS 13
    blur.inputImage = input.clampedToExtent() // no fading edges
    blur.radius = 8
    let tone = CIFilter.sepiaTone() // filters: NOT thread-safe
    tone.inputImage = blur.outputImage; tone.intensity = 0.6
    guard let out = tone.outputImage?.cropped(to: input.extent)
    else { return nil } // still only a recipe
    return context.createCGImage(out, from: out.extent) // GPU work: HERE
}
```

Custom colour kernel (MSL) + applying it

```
#include <CoreImage/CoreImage.h> // Threshold.ci.metal
extern "C" float4 threshold(coreimage::sample_t s, float t) {
    float l = dot(s.rgb, float3(0.2126, 0.7152, 0.0722)); // luma
    return float4(float3(step(t, l)), s.a); // black or white
} // Swift side:
let k = try CIColorKernel(functionName: "threshold",
                           fromMetalLibraryData: metalLibData)
let bw = k.apply(extent: img.extent, arguments: [img, 0.5])
```

Live camera — stay on the GPU

`AVCaptureVideoDataOutput → CVPixelBuffer → CIImage(cvpixelBuffer:) → filters → context.render` into the `MTKView` drawable's texture (or a `CIRenderDestination`) — never `createCGImage` per frame: that is a GPU→CPU readback plus an allocation, 30–60 times a second.

Which tool for an image effect?

Core Image	built-ins, graph optimisation, photo + video, RAW
SwiftUI shader	one view's pixels (<code>.colorEffect / .layerEffect</code> , iOS 17)
Metal	full control: your own passes, compute, 3D, ML
Core Graphics	drawing shapes and text, not filtering pixels

Traps

- ! A new `CITexture` per image or per frame — kernel recompiles, memory spikes, stutter.
- ! `UIImage(ciImage:)` in a `UIImageView`: still a recipe, rendered on every draw by a context you do not control. Render to a `CGImage` with your shared context.
- ! Soft transparent borders after a blur, or `createCGImage(out, from: out.extent)` is nil (infinite extent) — clamp, then crop.
- ! Rotated photo — `CIImage(image:)` drops the orientation.
- ! One `CIFilter` shared between threads — a data race; share images and the context instead.
- ! Face box drawn in the wrong place — Vision is normalised and bottom-left, UIKit top-left.

Remember

"CIImage is the recipe, CITexture is the kitchen — build the kitchen once, cook only what's ordered."